Добавь расчёт метрик и отдельно формирование отчета

Добавим два блока: **расчёт метрик** и **формирование отчёта** (текстовый + PDF). Это сделает модель не просто «симуляцией», а **инструментом анализа**.

Что добавим

- Расчёт метрик: среднее, стандартное отклонение, минимум, максимум, процентиль, число «опасных» точек.
- Сохранение метрик в `metrics_summary.csv`.
- Формирование текстового отчёта `report.txt`.
- Формирование PDF-отчёта `report.pdf` с графиками.

Обновлённый код

```
import os
import sys
import subprocess
import importlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import animation
from scipy.integrate import solve_ivp
from dataclasses import dataclass, field
from typing import List, Tuple
from fpdf import FPDF

# ------------------------------
# Проверка и установка библиотек
# ------------------------------

REQUIRED = ['numpy', 'scipy', 'matplotlib', 'pandas', 'fpdf']

def ensure_dependencies():
    for pkg in REQUIRED:
        try:
            importlib.import_module(pkg)
        except ImportError:
            print(f"Устанавливаю {pkg}...")
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', pkg])

ensure_dependencies()
```

```
# ------------------------------
# Параметры модели
# ------------------------------

@dataclass
class ModelParams:
    lambda_: np.ndarray = field(default_factory=lambda: np.array([0.5]*6))
    alpha: np.ndarray = field(default_factory=lambda: np.array([0.2]*6))
    beta: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    gamma: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    delta: np.ndarray = field(default_factory=lambda: np.array([0.05]*6))
    theta: np.ndarray = field(default_factory=lambda: np.array([0.3]*6))
    tau_E: float = 2.0

# ------------------------------
# Ядро модели
# ------------------------------

class LivingEquilibriumModel:
    def __init__(self, params: ModelParams):
        self.p = params

    def errors(self, x):
        return np.maximum(0, self.p.theta - x)

    def derivative(self, t, x):
        S, O, M, P, A, D = x
        lam, alpha, beta, gamma, delta, theta = (
            self.p.lambda_, self.p.alpha, self.p.beta,
            self.p.gamma, self.p.delta, self.p.theta
        )
        e = self.errors(x)
        x_bar = np.mean(x[:5])

        dS = -lam[^0]*S + alpha[^0]*e[^0] + beta[^0]*O*M - gamma[^0]*P - delta[^0]*(1
        dO = -lam[^1]*O + alpha[^1]*e[^1] + beta[^1]*S*M - gamma[^1]*P - delta[^1]*(1
        dM = -lam[^2]*M + alpha[^2]*e[^2] + beta[^2]*S*O - gamma[^2]*P - delta[^2]*(1
        dP = -lam[^3]*P + alpha[^3]*e[^3] + beta[^3]*S*O*M - gamma[^3]*A - delta[^3]
        dA = -lam[^4]*A + alpha[^4]*e[^4] + beta[^4]*P - gamma[^4]*(S+O+M) + delta[^4
        dD = -lam[^5]*D + alpha[^5]*e[^5] + beta[^5]*(S+O+M) - gamma[^5]*x_bar

        return [dS, dO, dM, dP, dA, dD]

# ------------------------------
# Запуск симуляции
# ------------------------------

def run_simulation():
    params = ModelParams()
    model = LivingEquilibriumModel(params)
    x0 = np.array([0.8, 0.8, 0.8, 0.8, 0.8, 0.8])
    t_span = (0, 20)
    t_eval = np.linspace(*t_span, 500)

    sol = solve_ivp(model.derivative, t_span, x0, t_eval=t_eval, method='RK45')
```

```
    return sol

# ----------------------------
# Сохранение в CSV
# ----------------------------

def save_results_to_csv(sol, filename='output/model_results.csv'):
    df = pd.DataFrame({
        'time': sol.t,
        'S': sol.y[^0],
        'O': sol.y[^1],
        'M': sol.y[^2],
        'P': sol.y[^3],
        'A': sol.y[^4],
        'D': sol.y[^5],
    })
    df.to_csv(filename, index=False)
    print(f"Результаты сохранены в {filename}")
    return df

# ----------------------------
# Расчёт метрик
# ----------------------------

def calculate_metrics(df, params):
    metrics = {}
    for col in ['S', 'O', 'M', 'P', 'A', 'D']:
        metrics[col] = {
            'mean': df[col].mean(),
            'std': df[col].std(),
            'min': df[col].min(),
            'max': df[col].max(),
            'q25': df[col].quantile(0.25),
            'q50': df[col].quantile(0.50),
            'q75': df[col].quantile(0.75),
            'danger_points': int((df[col] < params.theta[^0]).sum()),
        }
    return metrics

def save_metrics_to_csv(metrics, filename='output/metrics_summary.csv'):
    df = pd.DataFrame(metrics).T
    df.to_csv(filename)
    print(f"Метрики сохранены в {filename}")

# ----------------------------
# Формирование отчёта
# ----------------------------

def generate_text_report(metrics, filename='output/report.txt'):
    with open(filename, 'w', encoding='utf-8') as f:
        f.write("ОТЧЁТ ПО МОДЕЛИ ЖИВОГО РАВНОВЕСИЯ\n")
        f.write("="*40 + "\n\n")
        for k, v in metrics.items():
            f.write(f"Контур {k}:\n")
            for mk, mv in v.items():
                f.write(f"  {mk}: {mv:.4f}\n")
```

```
            f.write("\n")
    print(f"Текстовый отчёт сохранён в {filename}")

def generate_pdf_report(metrics, sol, filename='output/report.pdf'):
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
    pdf.set_font("Helvetica", 'B', 16)
    pdf.cell(0, 10, "Отчёт по модели живого равновесия", ln=True, align='C')
    pdf.ln(10)

    pdf.set_font("Helvetica", size=12)
    for k, v in metrics.items():
        pdf.cell(0, 10, f"Контур {k}:", ln=True)
        for mk, mv in v.items():
            pdf.cell(0, 8, f"  {mk}: {mv:.4f}", ln=True)
        pdf.ln(5)

    # Вставка графика
    pdf.add_page()
    pdf.image('output/model_dynamics_2d.png', x=10, y=30, w=180)
    pdf.output(filename)
    print(f"PDF-отчёт сохранён в {filename}")

# ------------------------------
# Визуализация
# ------------------------------

def plot_2d(sol):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    plt.figure(figsize=(12, 6))
    for i in range(6):
        plt.plot(sol.t, sol.y[i], label=labels[i])
    plt.axhline(0, color='r', linestyle='--')
    plt.xlabel('Время')
    plt.ylabel('Значение')
    plt.title('Динамика модели живого равновесия')
    plt.legend()
    plt.grid(True)
    plt.savefig('output/model_dynamics_2d.png')
    plt.close()

def plot_3d(sol):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.plot(sol.y[^0], sol.y[^1], sol.y[^2], lw=2)
    ax.set_xlabel('S')
    ax.set_ylabel('O')
    ax.set_zlabel('M')
    ax.set_title('3D фазовый портрет: S-O-M')
    plt.savefig('output/phase_portrait_3d.png')
    plt.close()

def animate(sol):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    fig, ax = plt.subplots()
```

```
    lines = [ax.plot([], [], label=label)[^0] for label in labels]
    ax.set_xlim(0, len(sol.t))
    ax.set_ylim(0, 1)
    ax.legend()

    def update(frame):
        for i, line in enumerate(lines):
            line.set_data(sol.t[:frame], sol.y[i][:frame])
        return lines

    ani = animation.FuncAnimation(fig, update, frames=len(sol.t), interval=50, blit=
    plt.show()

# ------------------------------
# Главная функция
# ------------------------------

def main():
    os.makedirs('output', exist_ok=True)
    sol = run_simulation()

    # Сохранение сырых данных
    df = save_results_to_csv(sol, 'output/model_results.csv')

    # Расчёт и сохранение метрик
    params = ModelParams()
    metrics = calculate_metrics(df, params)
    save_metrics_to_csv(metrics, 'output/metrics_summary.csv')

    # Формирование отчётов
    generate_text_report(metrics, 'output/report.txt')
    generate_pdf_report(metrics, sol, 'output/report.pdf')

    # Визуализация
    plot_2d(sol)
    plot_3d(sol)
    animate(sol)

if __name__ == "__main__":
    main()
```

Что появится в `output/`

- `model_results.csv` — сырые данные по времени и контурам.
- `metrics_summary.csv` — сводные метрики по каждому контуру.
- `report.txt` — текстовый отчёт.
- `report.pdf` — PDF-отчёт с графиком.
- `model_dynamics_2d.png` — 2D-график.
- `phase_portrait_3d.png` — 3D-график.

Что делают метрики

- `mean`, `std`, `min`, `max` — базовая статистика.
- `q25`, `q50`, `q75` — квартили.
- `danger_points` — сколько раз контур опускался ниже порога.

Это стандартный набор из `pandas.DataFrame.describe`, дополненный «опасными точками». [1] [2] [3]

Финальная формулировка

> **Метрики — это не «цифры ради цифр», а способ увидеть, где система держится, а где начинает разрушаться.**

Отчёт — это способ **зафиксировать выводы**, чтобы их можно было передать, проверить и использовать дальше. [4] [5] [6]

⁂

1. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html
2. https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.describe.html
3. https://pandas.pydata.org/docs/getting_started/intro_tutorials/06_calculate_statistics.html
4. http://gabors-data-analysis.com/ai-course/da-knowledge/pdf-guide.html
5. https://www.nutrient.io/blog/top-10-ways-to-generate-pdfs-in-python/
6. https://www.reportlab.com/docs/reportlab-userguide.pdf
7. https://www.victorianweb.org/victorian/philosophy/mill/liberty.html
8. https://en.wikipedia.org/wiki/On_Liberty
9. https://pandas.pydata.org/docs/dev/reference/api/pandas.DataFrame.describe.html
10. https://gabors-data-analysis.com/da-w-ai-book/chapters/22-pdf-guide.html
11. https://pandas.pydata.org/pandas-docs/stable/dev/reference/api/pandas.DataFrame.describe.html
12. https://stackpractices.com/recipes/generate-pdf-report-python/
13. https://skillsmp.com/creators/hkuds/agentpilot/openspace-benchmarks-gdpval-skills-pdf-checklist-workflow-enhanced-a53b26
14. https://skillsmp.com/ko/creators/hkuds/openspace/benchmarks-gdpval-skills-pdf-checklist-workflow-enhanced-a53b26
15. https://skillsmp.com/fr/creators/hkuds/openspace/benchmarks-gdpval-skills-pdf-checklist-generation
16. https://www.leoniemonigatti.com/blog/pdf-report.html
17. https://github.com/kiruthikaaruljothiesrap/Automated-Report-Generation
18. https://www.britannica.com/print/article/624300
19. https://www.annenbergclassroom.org/glossary_term/majority-rule-and-minority-rights/
20. https://www.britannica.com/topic/veche